

A Distributed Framework for Continuous Online Fine-Tuning of Large Language Models (CSCI-566 Final Report)

Rohan Sharma Akshay Mathur Manas Tiwari

(rsharma7@usc.edu) (akshayma@usc.edu) (manastiw@usc.edu)

Sparsh Gupta Aaradhya Goyal Partha Yashraj

(sparshg@usc.edu) (aaradhya@usc.edu) (pyashraj@usc.edu)

Abstract

Consumer-facing large language models are frozen at deployment, leading to progressive knowledge decay in fast-moving domains. Retraining these models on updated data requires expensive offline batch pipelines that interrupt serving and do not scale to continuous data streams. We present a distributed online fine-tuning framework that transforms the conventional batch fine-tuning workflow into a continuous streaming process. Three independently executing services coordinate through Apache Kafka to deliver continuous model adaptation without service interruption: a data producer, a streaming LoRA trainer, and an inference server with live weight hot-swapping. LoRA’s parameter-efficient updates keep adapter weights at 5–20 MB, enabling full weight transfers over the message bus in seconds. We evaluate the framework on IMDb sentiment classification and E2E NLG structured data-to-text generation. On IMDb, distilgpt2 converges from 0% to 82.65% accuracy within 1,563 streaming steps, achieving an MCC of 0.69 with a final forgetting-max of 0.002. On E2E NLG, mean slot coverage climbs from 0.20 to 0.90, with the familyFriendly boolean inversion rate declining from 15.8% to 1.2%. A multi-dimensional evaluation suite spanning classification metrics, temporal retention indicators, and qualitative proxy strategies provides visibility into online adaptation dynamics that standard endpoint reporting would miss.

1 Introduction

The conventional large language model lifecycle follows a rigid sequence: collect a static dataset, train offline, deploy, and repeat the entire process when the training data becomes stale. While this paradigm suffices for benchmark evaluation, it introduces significant operational limitations in production settings:

- The deployed model cannot incorporate information that appeared after its training cutoff. As terminology evolves and user behavior shifts, the model’s outputs become progressively less relevant.
- Retraining requires either duplicating the serving

infrastructure or taking the production endpoint offline during the update cycle.

There has been a growing interest in **continual learning** and **online adaptation** as potential remedies. Recent surveys (Shi et al., 2024; Chen et al., 2024) highlight the promise of sequential updates but also flag a core dilemma: absorbing new information can degrade previously learned capabilities, a phenomenon known as catastrophic forgetting (De Lange et al., 2021; Lesort et al., 2020). At the systems level, the need for asynchronous data handling aligns naturally with distributed streaming architectures, while parameter-efficient fine-tuning (PEFT) methods such as LoRA (Hu et al., 2022) and QLoRA (Dettmers et al., 2023) make the weight-update granularity small enough to be practical over a network transport.

We present a distributed framework that integrates these advances into a unified system. Rather than requiring the full dataset to be assembled before training begins, the proposed framework ingests training samples from an Apache Kafka stream, updates a LoRA adapter incrementally as data arrives, and propagates the updated adapter weights to a live inference server in the background—without restarts or downtime. Since LoRA adapters typically constitute less than 1% of total model parameters, the serialization-transfer-application cycle completes in seconds even for multi-billion parameter base models.

Our contributions are threefold:

1. A fully decoupled, streaming fine-tuning architecture built on Apache Kafka, LoRA, and HuggingFace Transformers that is entirely config-driven and requires no code changes to support new datasets or models.
2. A multi-dimensional evaluation framework spanning quantitative metrics (accuracy, F1, MCC, kappa, AUC, backward transfer, forgetting, perplexity) and qualitative proxy strategies (semantic similarity, keyword density, structured slot coverage) that provides longitudinal visibility into online adaptation dynamics.
3. Empirical validation across multiple datasets and model architectures demonstrating stable convergence and consistent system throughput under continuous streaming conditions.

2 Related Work

The proposed framework draws on advances in three interrelated research areas: parameter-efficient fine-tuning, online learning theory, and distributed data streaming systems.

Parameter-Efficient Fine-Tuning. Full fine-tuning of modern LLMs is prohibitively expensive for continuous updates. LoRA (Hu et al., 2022) addresses this by injecting trainable low-rank matrices into frozen model layers, reducing trainable parameters by roughly three orders of magnitude while maintaining task performance. QLoRA (Dettmers et al., 2023) extends this approach with 4-bit quantization of the base model, cutting GPU memory usage by over 60% compared to full fine-tuning. Recent survey work (Han et al., 2024) positions these methods within a broader taxonomy of adapter, prompt-tuning, and sparse fine-tuning strategies. MixLoRA (Li et al., 2024b) integrates multiple LoRA experts in feed-forward networks for multi-task settings. Our framework builds directly on LoRA and leverages the small adapter footprint as the key enabler of its streaming weight transfer mechanism.

Online and Continual Learning. The theoretical underpinnings of incremental model updates are well-established in the online learning literature (Hoi et al., 2018), which formalizes tradeoffs between exploration and exploitation under streaming data conditions. Continual learning extends these ideas to deep networks, where the primary challenge is catastrophic forgetting: the tendency of gradient updates on new data to overwrite previously learned representations (De Lange et al., 2021; Lesort et al., 2020). Gradient Episodic Memory (Lopez-Paz and Ranzato, 2017) and related methods attempt to preserve past knowledge through replay buffers, while Online LoRA (Wei et al., 2024) proposes continual adaptation of low-rank adapters specifically. Surveys on continual learning for LLMs (Shi et al., 2024; Wang et al., 2024) emphasize the need for evaluation protocols that go beyond final accuracy and capture temporal dynamics, knowledge retention, and update-level behavior.

Distributed stream systems. Apache Kafka (Kreps, 2011) provides a durable, replayable message log with built-in backpressure management, making it a natural backbone for real-time data pipelines. Its consumer group model allows multiple independent downstream services to process the same data stream without coordination overhead. Our framework uses Kafka in a dual role: as the training data transport (producer to trainer) and as the weight update channel (trainer to inference server). While Kafka’s messaging semantics are well-understood in the data engineering community, applying them to the specific problem of streaming weight-update distribution for LoRA adapters is, to our knowledge, a novel systems-level integration.

3 Problem Description

Modern LLMs rely on the Transformer architecture (Vaswani et al., 2023), which enables self-

supervised training on massive unlabeled corpora. While this produces models with impressive emergent abilities, the resulting weights are frozen at deployment time. The model does not learn from user interactions, and there is no built-in mechanism for incorporating new information after training concludes (Brown et al., 2020).

This static nature creates a concrete operational problem. Since real-world information constantly evolves, a model’s outputs inevitably lose relevance over time (Lazaridou et al., 2021). A model trained on data from one period will not be able to discuss events, terminology, or shifts in user behavior that occurred after its cutoff date.

Addressing this requires a method for continuous adaptation. Fine-tuning offers a practical path, keeping most parameters frozen and updating only specific layers, but continuous deployment introduces three specific challenges:

1. **Cost:** Even with parameter-efficient methods, continuous updates are non-trivial. Methods like LoRA mitigate this substantially, but the training infrastructure must still support frequent forward-backward passes under real-time throughput constraints.
2. **Data format:** Traditional training operates on static, pre-assembled datasets where the full corpus is available before training begins. In a continuous setting, data arrives as an unbounded stream with no predetermined end. The training pipeline must therefore assemble variable-length mini-batches incrementally from this real-time source, handling backpressure and variable arrival rates.
3. **Inference isolation:** Updates must not interfere with a live inference endpoint. The training process, the data source, and the serving infrastructure must operate independently, with a synchronization mechanism for propagating new weights that does not require downtime.

The proposed framework addresses all three challenges through a decoupled architecture where data ingestion, model training, and inference are independent services connected only through message-passing via Kafka.

4 Methods

4.1 System Architecture

The framework separates concerns into three loosely-coupled services, each running as an independent process (Figure 1).

The entire system is driven by a single YAML configuration file. Switching between datasets, models, or hyperparameter configurations requires no code changes—only a new configuration file. This design substantially reduces the overhead of running experiments across different task-model combinations.

Producer (`producer.py`) loads a HuggingFace dataset, applies configurable quality filtering through a

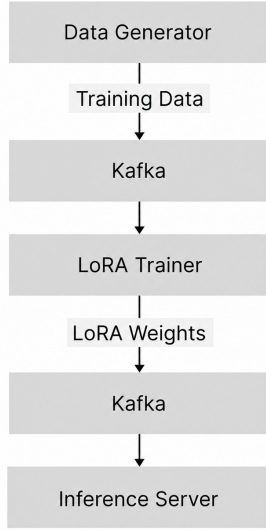


Figure 1: System architecture. Three services communicate exclusively through Kafka topics. The evaluator supports both inline and decoupled offline evaluation against saved checkpoints.

two-tier short-circuit pipeline (length checks, alphanumeric ratio, zlib compression-based repetition detection, and optional regex patterns), and publishes each sample as a JSON record to a Kafka training-data topic. Message keys are SHA-256 hashes of the input text, enabling broker-side deduplication and deterministic partition routing.

Trainer (`trainer.py`) consumes training records from Kafka, tokenizes them with label masking (prompt tokens are masked so that only response tokens contribute to the cross-entropy loss), assembles variable-length mini-batches with dynamic padding, and trains a LoRA adapter using AdamW with gradient accumulation. After every configurable interval, the trainer serializes the LoRA adapter state dictionary and publishes it layer-by-layer to a Kafka `lora-updates` topic. Checkpoints are saved to disk at regular intervals with full provenance metadata.

Inference Server (`inference.py`) loads the same base model with an identical LoRA configuration and exposes a Flask REST API. A background Kafka consumer thread deserializes incoming weight tensors into a queue. A separate weight-application thread drains the queue in a batch, acquires a model lock, and applies all accumulated updates in a single `load_state_dict` call. This batching strategy minimizes lock contention: if the trainer pushes ten tensor updates rapidly, they are applied in one lock acquisition rather than ten sequential ones.

Evaluator (`evaluate.py`) supports two modes of operation. During training, the framework can perform inline evaluation at configurable step intervals for real-time monitoring of model convergence. In addition, a decoupled offline evaluation mode allows loading any saved LoRA checkpoint and running the full evaluation suite (both quantitative and qualitative) without requiring a Kafka connection. This decoupled mode was introduced because inline qualitative evaluation—

particularly structured slot coverage with multiple consistency runs—could block training ingestion for 15–40 minutes per evaluation interval, which is impractical for real-time streaming. The decoupled evaluator enables more comprehensive testing at the experimenter’s convenience without affecting training throughput.

4.2 LoRA Adapter Training

LoRA (Hu et al., 2022) replaces the full weight update with two low-rank decomposition matrices:

$$W' = W + \frac{\alpha}{r}(B \times A) \quad (1)$$

The base model remains frozen; only the low-rank matrices A and B are trained. Because the adapter constitutes a small fraction of total model parameters, the resulting state dictionary is compact enough for efficient serialization and transfer over a message bus. In our experiments, we use $r = 16$ and $\alpha = 32$. The target modules for LoRA injection are configurable and can be adjusted based on the model architecture and task requirements. For GPT-2 family models, we target `c_attn` and `c_proj`.

4.3 Tokenization and Label Masking

Each training sample is tokenized as a concatenation of prompt and response:

$$\text{input_ids} = [\text{prompt_ids} \parallel \text{response_ids} \parallel \text{EOS}]$$

Labels mirror `input_ids` but mask all prompt positions so that only the response tokens contribute to the cross-entropy loss. This masking is implemented using the HuggingFace `ignore index` convention, ensuring the model learns to produce the response content rather than memorizing the prompt template. The masking boundary is determined dynamically based on the tokenized prompt length and is configurable through the training configuration. The response is never truncated; if the combined sequence exceeds the configured maximum sequence length, the prompt is truncated from the end.

4.4 Training Loop

The main loop polls Kafka for messages, assembles mini-batches of size B , computes the forward pass, scales the loss by $1/G$ (where G is the gradient accumulation factor), and calls `backward()`. Every G micro-batches, gradient norms are clipped to a maximum norm of 1.0 before the AdamW step. To maintain stable memory utilization under continuous operation, the training loop performs explicit memory cleanup after each optimizer step, including tensor deallocation, garbage collection, and device-specific cache clearing. This memory management strategy was necessary to prevent PyTorch’s computational graph buffers from accumulating during Kafka poll wait periods, which on Apple Silicon devices could lead to memory exhaustion and gradient instability.

Learning rate scheduling uses a `cosine_with_warmup` policy:

$$\lambda(t) = \begin{cases} t/t_w & \text{if } t < t_w \\ \mu + (1 - \mu) \cdot \frac{1 + \cos(\pi \cdot \frac{t - t_w}{T - t_w})}{2} & \text{otherwise} \end{cases}$$

where t_w is the warmup duration, T is the scheduler period, and μ is the minimum LR ratio floor.

4.5 Evaluation Framework

4.5.1 Quantitative Metrics

The framework supports multiple evaluation strategies depending on the task:

- **Class match:** for classification tasks (e.g., IMDb sentiment). Generated text is prefix-matched against the known label set. Off-label outputs are captured in an “other” bucket to prevent confusion matrix corruption.
- **Perplexity:** for open-ended generation tasks. Only forward-pass loss is computed.

Beyond task-level accuracy, the evaluator computes temporal and retention metrics drawing on the continual learning literature (Wang et al., 2024; Lopez-Paz and Ranzato, 2017):

- **AAUC** (Average Accuracy Under the Curve): normalized trapezoidal area under the accuracy-vs-step curve, capturing cumulative learning dynamics rather than only final performance.
- **Backward Transfer:** for each sample currently predicted incorrectly, checks if it was correct at a previous evaluation step. Negative BWT indicates regression on previously learned examples.
- **Forgetting-Max:** maximum drop from peak value across tracked metrics. Meaningful under stable evaluation pools.

4.5.2 Qualitative Proxy Strategies

Three qualitative strategies assess generation quality dimensions that standard metrics cannot capture:

- **Semantic similarity:** cosine similarity between generated and golden response embeddings via sentence-transformers/all-MiniLM-L6-v2 (CPU-only, 90 MB). Used for instruction-following tasks.
- **Keyword density:** fraction of a curated domain keyword list present in the generation, plus type-token ratio and hapax ratio for lexical diversity. Reference-free. Used for domain adoption (IMDb qualitative).
- **Structured slot coverage:** parses meaning representations into slot-value pairs, checks each against the generated text (including boolean negation handling for yes/no attributes), and measures coverage, consistency across stochastic runs, and per-slot learning curves. Used for data-to-text NLG (E2E).

All strategies additionally compute universal quality indicators: mean response length, repetition rate (fraction of repeated bigrams), and non-empty rate.

4.6 Configuration System

All services and the evaluator share a single YAML configuration file. The system is entirely config-driven: new datasets and models can be onboarded by writing a new YAML file without modifying any source code. Multiple reference configurations covering classification, instruction following, and structured NLG tasks are provided with the framework.

5 Experimental Setup

5.1 Datasets and Tasks

We evaluate the framework on two datasets spanning different task categories. Table 1 summarizes the experimental configurations.

Dataset	Task	Model	Strategy
IMDb	Sentiment	distilgpt2	class_match
E2E NLG	Slot-to-text	gpt2-medium	perplexity

Table 1: Experimental configurations across the primary evaluation datasets.

IMDb Sentiment Classification (primary experiment). The IMDb movie review dataset contains 25,000 training and 25,000 test samples with binary sentiment labels. We use distilgpt2 (82M parameters) with LoRA rank 16, $\alpha = 32$, targeting the `c_attn` and `c_proj` attention layers. Training uses batch size 4 with 4 gradient accumulation steps (effective batch size 16), learning rate 1×10^{-4} with cosine warmup over 50 steps, and max sequence length 512 tokens. Evaluation uses the `class_match` strategy on a pool of 4,000 held-out test samples, evaluating the full pool at each checkpoint.

E2E NLG Structured Generation. The E2E NLG Challenge dataset (42,061 train, 4,672 validation) tests structured data-to-text generation, where the input is a meaning representation (MR) consisting of slot-value pairs (e.g., `name[The Punter]`, `eatType[coffee shop]`, `priceRange[less than £20]`) and the expected output is a fluent restaurant description that faithfully verbalizes all specified attributes. We use gpt2-medium (345M parameters) with LoRA rank 16, $\alpha = 32$, and the `structured_slot_coverage` qualitative strategy. This strategy parses the MR into individual slots, checks each against the generated text (including boolean negation handling for attributes such as `familyFriendly`), and measures per-slot coverage, consistency across multiple stochastic generation runs, and per-slot learning curves over training steps.

5.2 Hardware and Software

All experiments were conducted on consumer-grade hardware: Apple Silicon M-series machines (16 GB unified memory) and NVIDIA laptop GPUs. The software stack comprises PyTorch (with MPS/CUDA backend auto-detection), HuggingFace Transformers, PEFT, kafka-python, and Flask. Apache Kafka runs in KRaft mode with no Zookeeper dependency.

5.3 Training Protocol

All reported experiments use a single-pass streaming protocol designed to evaluate the framework under controlled conditions. At startup, the trainer seeks to the end of the Kafka topic so that it processes only records from the current producer run, ensuring no contamination from prior experiments. Training continues until the producer sends an EOF sentinel, guaranteeing exactly one pass through the full streaming dataset. This protocol is equivalent to a single epoch of streaming training. To prevent offset collisions across restarts, the trainer uses a UUID-suffixed consumer group identifier.

6 Results and Discussion

We report results from two fully evaluated experiments: IMDb sentiment classification (quantitative, decoupled offline evaluation) and E2E NLG (qualitative slot coverage).

6.1 Evaluation Loss and Perplexity

Using the decoupled offline evaluator against the full 4,000-sample test pool, evaluation loss (Figure 2) drops sharply from 6.16 at step 0 (base model, no fine-tuning) to 0.35 by step 50, reaching 0.12 by step 600. After step 600 the loss stabilizes in the 0.12–0.13 range through the final checkpoint at step 1,563. The corresponding perplexity curve (Figure 3) mirrors this trajectory, decreasing from 474.6 (base model) to 1.42 at step 50, then converging toward 1.12 by step 1,250 and remaining there.

The reduction in perplexity from 474.6 to 1.12 is notable, indicating that the model transitions from near-random prediction to confident sentiment classification within fewer than 1,600 streaming steps. Perplexity is computed over the response tokens only (i.e., the classification label), consistent with the label masking strategy described in Section 4.3. The rapid convergence within the first 50 steps, followed by gradual refinement, suggests that the LoRA adapter efficiently captures the task-relevant signal from the streaming data. For context, standard offline fine-tuning of distilgpt2 on IMDb with multiple epochs typically achieves 88–90% accuracy; the streaming approach reaches 82.65% in a single pass, trading peak accuracy for continuous adaptability without service interruption.

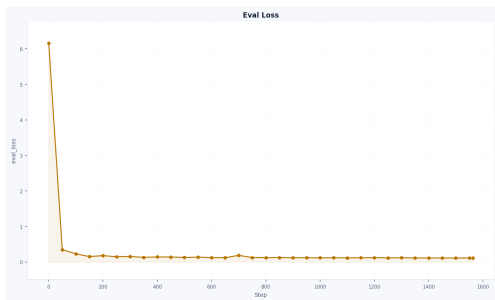


Figure 2: Evaluation loss over training steps (decoupled offline evaluation, full 4,000-sample pool). Loss drops from 6.16 to below 0.15 within the first 150 steps and stabilizes near 0.12.

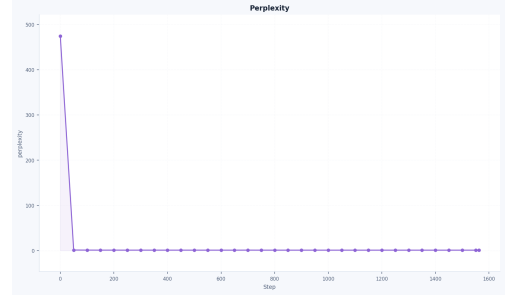


Figure 3: Perplexity over training steps. Drops from 474.6 (base model) to approximately 1.12 and remains stable.

6.2 Classification Accuracy and Correlation Metrics

Classification accuracy (Figure 4) rises from 0% at step 0 to 50.8% at step 50, then climbs steeply to 74.0% at step 100 and 78.8% at step 150. Accuracy crosses 80% by step 250 and stabilizes in the 80–83% range for the remainder of training, with a final value of 82.65% at step 1,563. The convergence pattern shows rapid early learning followed by a stable plateau, consistent with the LoRA adapter reaching the representational ceiling for this model-task combination under single-pass streaming.

Macro F1 (Figure 5) follows a similar trajectory, rising from 0.356 at step 50 to 0.522 at step 100 and stabilizing near 0.576 by step 600. The MCC (Figure 6) provides a more discriminative measure: starting at 0.127 at step 50, it jumps to 0.543 at step 100, reaches 0.684 by step 600, and converges at 0.689 by step 1,563. MCC is particularly informative here because it accounts for all four confusion matrix quadrants, confirming that the model genuinely discriminates between positive and negative sentiment rather than defaulting to majority-class prediction. Cohen’s kappa tracks MCC closely, settling at 0.68.

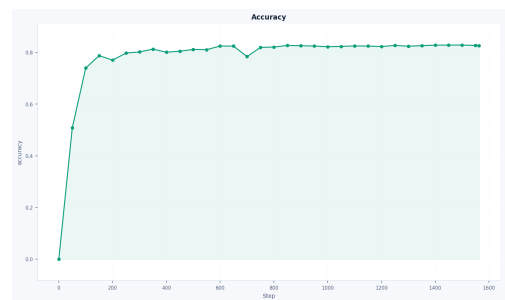


Figure 4: Classification accuracy on the IMDb sentiment task (full-pool evaluation). Accuracy rises from 0% to above 80% within the first 250 steps and stabilizes near 82.65%.

6.3 Knowledge Retention

The forgetting-max metric (Figure 7) tracks the maximum drop from peak performance across all monitored metrics and shows a distinctive pattern. For the first 200 steps, forgetting is near zero as the model is only improving. A spike at step 200 (0.033) and a larger spike at

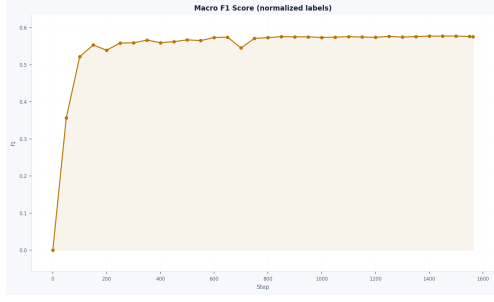


Figure 5: Macro F1 score over training steps. F1 converges from 0.356 at step 50 to 0.576 by step 600 and remains stable.

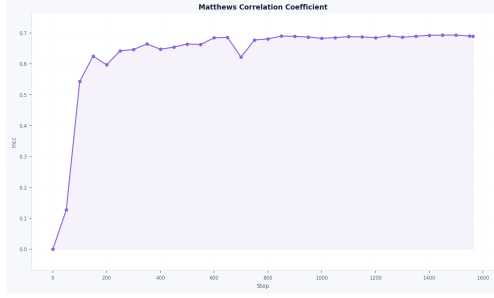


Figure 6: Matthews Correlation Coefficient over training steps. MCC reaches 0.69 by step 600, confirming balanced discrimination between sentiment classes.

step 700 (0.078) correspond to evaluation checkpoints where accuracy temporarily dipped below its running maximum, likely driven by the stochastic nature of streaming data ordering. After step 800, forgetting-max decays below 0.01 and remains there through the final checkpoint.

It is important to note that while the forgetting-max and backward transfer metrics provide indicators of knowledge retention within a single training run, we did not conduct formal catastrophic forgetting experiments (e.g., sequential multi-task training with replay). The metrics reported here measure within-task stability under streaming data ordering, not cross-task interference.

Backward transfer remains consistently negative (between -0.05 and -0.11), which in this metric’s convention indicates net improvement over time: samples that were previously predicted incorrectly are being learned, while very few previously correct samples are lost. The continual learning stability metric (Figure 8) corroborates this. The monotonically decreasing curve shows continuous improvement relative to the initial evaluation, with diminishing marginal gains after step 300 as the model saturates.

Average accuracy (AAUC, Figure 9) rises smoothly from 0.25 at step 50 to 0.79 at step 1,563, reflecting the cumulative area under the accuracy curve. This metric captures the full learning trajectory rather than only the endpoint, and its smooth upward trend confirms stable, monotonic improvement throughout training.

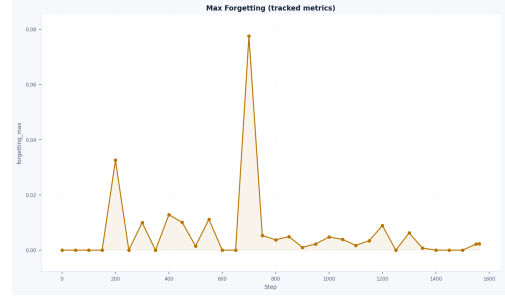


Figure 7: Forgetting-max over training steps. Spikes at steps 200 and 700 represent temporary accuracy dips; after step 800 forgetting remains below 0.01.

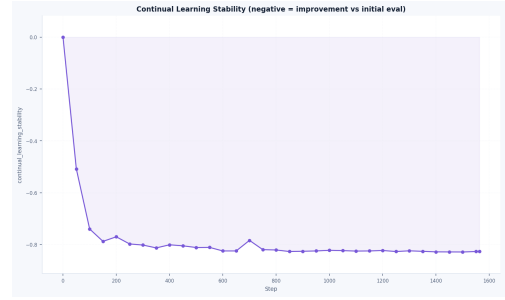


Figure 8: Continual learning stability (negative = improvement vs. initial evaluation). The monotonically decreasing curve confirms continuous net improvement throughout training.

6.4 E2E NLG: Structured Slot Coverage

The E2E NLG experiment provides our most detailed qualitative evaluation. Using gpt2-medium (345M parameters), the structured slot coverage strategy evaluates whether the model correctly verbalizes each attribute from the input meaning representation (MR).

Mean slot coverage (Figure 10) rises from 0.20 at step 0 (base model zero-shot) to 0.74 by step 200 and continues climbing to 0.90 by step 1,800, where it stabilizes. This represents a $4.5\times$ improvement in the model’s ability to faithfully convert structured data into natural language descriptions. Per-slot analysis reveals that high-salience attributes like name (0.97 coverage at step 200, 0.99 final) and eatType (0.90 at step 200, 0.97 final) are learned almost immediately, while lower-frequency attributes such as customer_rating ($0.05 \rightarrow 0.86$) and priceRange ($0.03 \rightarrow 0.75$) require significantly more training steps.

The familyFriendly boolean attribute provides a particularly informative signal. Around step 400, the model begins generating positive phrasing (e.g., “family-friendly”) even when the input specifies “no,” with the inversion rate peaking at 15.8% (Figure 11). This pattern suggests the model first learns attribute salience—recognizing that the slot is relevant—before learning its polarity. The inversion rate subsequently declines to 1.2% by the final checkpoint as the model resolves the semantic direction.

Consistency score (measuring agreement across stochastic generation runs for the same prompt) rises from 0.01 at step 0 to 0.63 at the final checkpoint, with

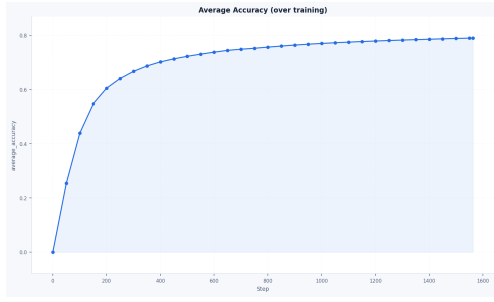


Figure 9: Average accuracy (AAUC) over training steps. Smooth rise to 0.79 captures the full learning trajectory rather than only endpoint performance.

perfect coverage rate climbing from 0% to 17.3%. The repetition rate remains below 1.4% throughout, confirming that the model generates diverse, non-degenerate text.

The modest increase in evaluation perplexity (59.4 $\rightarrow$ 70.2, Table 3) is consistent with the model learning to override its base distribution to include specific slot values that may have low prior probability, trading fluency at the token level for structured accuracy at the semantic level.

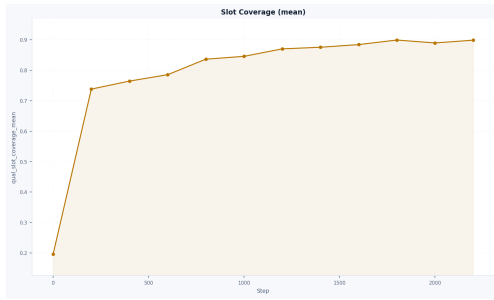


Figure 10: E2E NLG mean slot coverage over training steps. Coverage rises from 0.20 (zero-shot) to 0.90 by step 1,800.

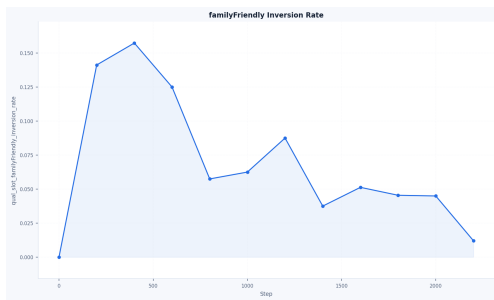


Figure 11: familyFriendly boolean inversion rate. Peaks at 15.8% (step 400) as the model learns attribute salience before polarity, then declines to 1.2%.

6.5 Engineering Observations

Several engineering decisions proved critical for stable operation.

Memory management on Apple Silicon. Without explicit memory cleanup (tensor deallocation, garbage collection, and cache clearing after each optimizer step),

PyTorch’s MPS backend accumulated 25+ GB of retained computational graph buffers during Kafka poll wait periods. The cleanup routine stabilizes memory at approximately 10 GB, making multi-task workloads viable on 16 GB machines.

Precision policy. FP16 on MPS under small batch sizes (2–8 samples) caused gradient overflow to NaN, corrupting AdamW momentum estimates permanently. Switching to FP32 as the default eliminated all NaN-related failures at no measurable quality difference for LoRA fine-tuning at these scales.

Batched qualitative generation. An important optimization for evaluation efficiency was the transition from sequential per-sample generation to batched generation. By processing samples in chunks of 32 with left-padding and GPU-parallel consistency runs (num_return_sequences), evaluation time was reduced from 40 minutes to under 4 minutes for the same 3,340 generation calls. This optimization was essential for making the decoupled evaluation workflow practical across multiple checkpoints.

7 Summary of Results

Tables 2 and 3 consolidate the key outcomes from our two fully evaluated experiments.

Metric	Step 0	Final (1563)
Eval Loss	6.16	0.12
Perplexity	474.6	1.12
Accuracy	0.0%	82.65%
Macro F1	0.0	0.576
MCC	0.0	0.689
Cohen’s Kappa	0.0	0.681
AAUC	0.0	0.79
Backward Transfer	—	−0.072
Forgetting-Max (peak)	0.0	0.078
Forgetting-Max (final)	0.0	0.002

Table 2: IMDb sentiment classification (decoupled of-line evaluation, full 4,000-sample pool). Backward transfer is negative, indicating net improvement over time.

Metric	Step 0	Final
Mean Slot Coverage	0.20	0.90
Perfect Coverage Rate	0.0%	17.3%
Consistency Score	0.01	0.63
familyFriendly Inversion	0.0%	1.2%
Repetition Rate	20.3%	1.0%
Eval Loss	4.08	4.25
Perplexity	59.4	70.2

Table 3: E2E NLG qualitative evaluation (gpt2-medium, structured slot coverage strategy). Eval loss/perplexity do not decrease because the qualitative configuration optimizes for generation structure, not next-token prediction.

8 Limitations and Future Work

Several limitations bound the current evaluation. Our primary quantitative experiment uses distilgpt2, a rela-

tively small (82M parameter) model; larger base models would test whether the streaming overhead and the Kafka-based weight transfer mechanism scale acceptably with model size. All experiments use a single-epoch streaming protocol with no replay, so multi-epoch streaming with catastrophic forgetting countermeasures (e.g., gradient episodic memory) remains unexplored.

Several directions for future work are worth noting. Implementing a Parameter Server architecture would address Kafka’s message size limitations and enable weight distribution for larger models. Docker containerization and Kubernetes orchestration would make the system deployable in cloud environments with auto-scaling and fault tolerance. Structured human evaluation and LLM-as-a-judge scoring (Li et al., 2024a; Kim et al., 2024) would complement the current proxy-based qualitative metrics with more direct assessments of response quality. Finally, integrating replay mechanisms or regularization-based continual learning strategies could extend the framework to multi-epoch settings where catastrophic forgetting becomes a more pressing concern.

Division of Labor

- **Rohan** led the implementation of the core continuous training loop and its integration with the Kafka consumer. He configured the Low-Rank Adaptation (LoRA) modules using the PEFT library and implemented the AdamW optimization step with customizable gradient accumulation. He also engineered the token masking mechanism, ensuring that prompt tokens are masked so they are ignored during the cross-entropy loss calculation, allowing the model to learn strictly from the response tokens. In addition, Rohan led the entire evaluation effort for the framework, including the design of the decoupled evaluation pipeline, the qualitative evaluation module with four proxy strategies, and the multi-dataset experimental analysis.
- **Akshay** focused on generalizing the trainer architecture to support multiple models and datasets programmatically. He implemented the configuration parsing system that extracts hyperparameters from YAML files and abstracted the model initialization using the HuggingFace AutoModelForCausalLM interface. Additionally, he developed the dynamic batch collation and padding logic, ensuring the trainer can process varying sequence lengths and formats without requiring hardcoded dimensions or dataset-specific rules.
- **Spارش** built the testing and evaluation suite for the training module. He implemented polymorphic evaluation strategies within the Evaluator class, including perplexity calculation and exact class matching to assess model convergence. Furthermore, he developed the monitoring system that incrementally writes training metrics to a CSV file and programmatically generates performance plots to track cross-entropy loss and accuracy over time.

- **Manas** developed the data generation pipeline, specifically building the Kafka producer to ingest and stream text data. He implemented the SHA-256 hashing mechanism applied to input text, which generates stable message keys to ensure deterministic partition routing and support broker-side deduplication. He also integrated the data quality filtering logic via the StreamFilter module, set up the dataset shuffling and column mapping logic, and conducted initial unit testing to verify message delivery and streaming throughput from the source to the Kafka broker.
- **Aaradhya** engineered the real-time weight update mechanism within the inference server. He implemented the background Kafka consumer thread that listens for serialized LoRA tensors, deserializes them from byte buffers, and safely injects the updated state dictionary into the live model using a threading lock to prevent data races. Alongside this architectural work, he researched quantitative testing methodologies to identify appropriate statistical metrics and benchmark datasets for evaluating the framework’s online learning efficiency.

- **Partha** developed the inference server’s REST API using Flask, establishing the primary POST /generate endpoint for handling user queries. He integrated the text generation logic with the base model, configuring constraints such as max_new_tokens and standardizing the JSON request and response formatting. In addition to the API development, he investigated qualitative evaluation frameworks, outlining comparative assessment strategies and prompt-response review criteria to measure output coherence and task alignment.

9 Conclusion

We have presented a distributed framework for continuous, stream-based fine-tuning of large language models. The system decomposes adaptation into three independently executing services (data ingestion, training, and inference) coordinated through Apache Kafka. LoRA’s parameter efficiency makes weight-update transfer practical at the message bus level: adapters are small enough to serialize, transmit, and apply in seconds.

Experimental evaluation on the IMDb sentiment task shows that streaming fine-tuning with single-pass data consumption achieves 82.65% classification accuracy, up from 0%, with an MCC of 0.69, AAUC of 0.79, and a final forgetting-max of 0.002 (peak 0.078). On the E2E NLG task, slot coverage rises from 0.20 to 0.90, with the boolean inversion rate declining from 15.8% to 1.2%, demonstrating that the framework supports qualitative generation tasks alongside classification. The multi-dimensional evaluation suite spanning accuracy, F1, MCC, kappa, AAUC, backward transfer, forgetting, perplexity, and three qualitative proxy strategies provides visibility into aspects of online adaptation that standard final-accuracy reporting would miss.

The framework is entirely config-driven: adding new datasets and models requires only a new YAML file, not code changes. Multiple reference configurations across several datasets demonstrate generalization of the approach to classification, instruction following, and structured NLG. The decoupled offline evaluation infrastructure, including hierarchical checkpoint management and automated artifact generation, supports systematic experimentation with continual learning dynamics without blocking the streaming training pipeline.

The results demonstrate that continuous LLM adaptation through streaming fine-tuning is practical on consumer-grade hardware with existing open-source tools. The architectural pattern of decoupled streaming services with parameter-efficient adapter transfer offers a viable alternative to the traditional batch retraining cycle for scenarios where model freshness is operationally important.

References

- Tom Brown, Benjamin Mann, Nick Ryder, and 1 others. 2020. [Language models are few-shot learners](#). *Advances in Neural Information Processing Systems (NeurIPS)*.
- Yifan Chen, Yiran Ge, and 1 others. 2024. Towards continual learning for large language models: A survey. *arXiv preprint*.
- Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Ales Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. 2021. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. [Qlora: Efficient finetuning of quantized llms](#). *arXiv preprint arXiv:2305.14314*.
- Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Shuo Zhang. 2024. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*.
- Steven C. H. Hoi, Doyen Sahoo, Jing Lu, and Peilin Zhao. 2018. [Online learning: A comprehensive survey](#). *Preprint*, arXiv:1802.02871.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, and 1 others. 2022. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3.
- Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and 1 others. 2024. Prometheus 2: An open source language model specialized in evaluating other language models. *arXiv preprint arXiv:2405.01535*.
- Jay Kreps. 2011. [Kafka : a distributed messaging system for log processing](#).
- Angeliki Lazaridou, Mayumi Villalba, and Nicola De Cao. 2021. [Pitfalls of static language modelling](#). *arXiv preprint arXiv:2102.01951*.
- Timothée Lesort, Vincenzo Lomonaco, Andrei Stoian, Davide Maltoni, David Filliat, and Natalia Díaz-Rodríguez. 2020. Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, 58:52–68.
- Dawei Li and 1 others. 2024a. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. *arXiv preprint arXiv:2411.16594*.
- Dengchun Li, Yingzi Ma, Naizheng Wang, Zhengmao Ye, Zhiyuan Cheng, Yinghao Tang, Yan Zhang, Lei Duan, Jie Zuo, Cal Yang, and Mingjie Tang. 2024b. [Mixlora: Enhancing large language models fine-tuning with lora-based mixture of experts](#). *Preprint*, arXiv:2404.15159.
- David Lopez-Paz and Marc’Aurelio Ranzato. 2017. Gradient episodic memory for continual learning. *Advances in Neural Information Processing Systems*, 30.
- Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyuan Wang, Yibin Wang, and Hao Wang. 2024. Continual learning of large language models: A comprehensive survey. *arXiv preprint arXiv:2404.16789*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. [Attention is all you need](#). *Preprint*, arXiv:1706.03762.
- Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. 2024. A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Xiwen Wei and 1 others. 2024. Online lora: Continual fine-tuning with online adaptation of low-rank adapters. *arXiv preprint*.